COMP 018: Database Administration — Activity 4 Progress Report

Access Control, Database Users, Roles, Privileges, and Security Governance

Project Identification & Metadata

Course	COMP 018 - Database Administration
Evaluating Group	Group 4: Olivarez, Charles Kean E., Infantado, Gerwin G., Pocsidio, Jhon Louie F., Abangan, SJ Kaizen M., Bartolata, Redge Raindel N.
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	BoardTrack: Boarding House & Rental Management System
Target Project Group	Group 11: Barcarse, Adrian; Cataluña, Zyruz Gabriel; Camu, Edgar; Policarpio, Kurt; Crucillo, Karl Robert
Target Project Repository	https://github.com/karlrobertc-art/BoardTrack
Primary Database	boardtrack_db (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	boardtrack_db_restore_test

ACTIVITY 4: Database Users, Roles, Privileges, and Access Control

1.1 Access-Control Plan & Architecture

In default database configurations, web backend applications frequently connect using an unrestricted superuser account. For Group 11's BoardTrack boarding house management platform, connecting landlord dashboards, tenant portals, and lease tracking tools using a database superuser account severely violates the Principle of Least Privilege. If an unauthenticated user were to exploit an application vulnerability, they would possess unrestricted data definition and modification capabilities across the cluster, threatening lease contracts, rental payment histories, and tenant security.

To establish secure operations for Group 11's BoardTrack platform, a role-based access control architecture was implemented. This architecture separates landlord operational lease management from administrative maintenance while strictly isolating tenant billing query workflows from property modifications. The structure utilizes group roles without login

capabilities to manage privilege collections, alongside individual login accounts assigned to matching group roles.

Under this access control architecture, the application runtime account holds selection, insertion, and update capabilities across all operational tables, while confining record deletion strictly to erroneous maintenance requests to support ticket cleanup. Row deletion remains strictly denied on master property records, rooms, tenant profiles, and verified payments. Read-only reporting and auditor accounts are granted query access across rooms, active leases, and financial summary views to audit rental performance, but are strictly denied access to user credential tables containing login credentials and password hashes. Logical backups are delegated to a dedicated backup role with schema-wide selection rights and zero write permissions.

1.2 Access-Control Role Definitions and Privilege Structure

The application runtime user operates under a dedicated group role assigned to handle property listings, room allocations, lease contracts, and monthly rental payments. Its permissions maintain full selection, insertion, and update capabilities on operational tables while confining deletion strictly to maintenance requests, protecting properties, rooms, tenant records, and payment logs from deletion.

The read-only auditor account operates through a compliance group role to review room occupancy, lease statuses, and dynamic billing summaries via relational views (v_lease_summary). It holds read access across operational tables while omitting read permissions on user credentials to safeguard authentication data. The backup operator account is assigned dedicated read permissions across tables and sequences to support logical dump generation without write or schema modification rights. Database administration tasks remain strictly reserved for the database administrator superuser account.

1.3 Access-Control Configuration Summary

The configuration process established three non-login group roles representing application runtime, read-only reporting, and backup operator profiles, followed by the creation of matching login accounts with dedicated authentication credentials. Access permissions were revoked from public roles across the database and schema to enforce explicit privilege assignment. Connection and schema usage rights were assigned to the designated group roles, specific table grants were applied according to operational requirements, and default schema privileges were configured to maintain access definitions for future objects.

1.4 Verification Results: Runtime Account Testing

Verification testing demonstrated that the application runtime account successfully reads property listings, registers newly signed leases while advancing sequence counters, updates room occupancy statuses, and records monthly rental payments. When deletion operations were attempted on master property assets, active room records, or tenant profiles, or when DDL commands were attempted, the database engine denied execution under the narrow deletion policy and object ownership constraints.

1.5 Verification Results: Read-Only Auditor Account Testing

Inspection testing confirmed that the reporting account successfully executes query searches across available rooms, property listings, and lease financial summaries via v_lease_summary. When querying user profile tables containing landlord passwords, access was denied by the database security layer. Attempted modification operations, including submitting unauthorized lease records, altering room rental rates, deleting payment histories, or querying credential records, were uniformly blocked by the database engine.

1.6 Verification Results: Cross-Database & Administrative Boundary Testing

Administrative isolation testing showed that database creation commands attempted under application accounts were denied due to missing creation privileges. Connection attempts to unrelated databases on the server instance failed because public connection permissions were revoked, restricting access strictly to authorized targets.

1.7 Issues, Fixes, and Feedback to Owners

During initial setup, default superuser connection settings in application bootstrap scripts were replaced with scoped least-privilege credentials. System prerequisites were documented regarding the btree_gist extension required to support temporal exclusion constraints preventing room double-booking.

Feedback provided to Group 11 commended the exemplary temporal modeling using the btree_gist exclusion constraint (ex_lease_no_overlap), automated room status synchronization triggers, and dynamic balance computation via v_lease_summary. Recommendations advised documenting extension installation prerequisites for cloud deployments, alongside adding composite indexes on payment foreign keys to optimize financial reporting.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 4)

Column Header	**Value**
Project	Group 11 - BoardTrack: Boarding House & Rental Management System
Repository cloned	https://github.com/karlrobertc-art/BoardTrack, Branch: main
Database setup result	Success (Initialized PostgreSQL 18.6 on port 5432, built 8 tables, exclusion constraint, triggers, view, sample data, and configured least-privilege roles).
Access-control folder link	https://github.com/Nycri/brewpos/tree/main/access-control
Access-control status	Completed

Feedback sent to owners	Yes (05/11/2026)
PDF progress document	https://github.com/Nycri/brewpos/blob/main/PDF_PROGRESS_DOCUMENTATION.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.